
# Docker

## Storage & Networking

Erfan Taheri

آکادمی
همراه‌اول

آکادمی همراه‌اول

# Storage - Overview

- Writable container layer atop read-only image layers (CoW via overlayfs)
- Ephemeral: lifecycle tied to container (deleted on removal)
- tmpfs provides memory-backed ephemeral storage (fast, volatile)
- Volumes and bind mounts provide persistence outside the container layer

آکادمی همراه‌اول

# OverlayFS, Writable Layer & Volumes (visual)

Layer A (base)

Layer B (runtime libs)

Layer C (app code)

Writable layer (upperdir - CoW)

Named Volume (host path /var/lib/docker/volumes/<id>/_data)

Persistent storage (bind to host or remote driver)

Docker mounts volume into container at specified mountpoint (overrides path in merged FS)

Merged mount (container rootfs view - overlayfs)

آکادمی همراه‌اول

# Docker Volumes

## What Are Volumes?

- Managed storage mechanism provided by Docker
- Stored **outside the container filesystem** (under /var/lib/docker/volumes/)
- Survive container restarts & removals

## Key Characteristics

- **Persistent** → data is not deleted when container stops
- **Portable** → can be shared across containers
- **Driver-based** → supports 3rd-party storage backends (NFS, Ceph, AWS EBS, etc.)
- Better performance & isolation than bind mounts

## Use Cases

- Databases (MySQL, PostgreSQL)
- Shared data between multiple containers
- Offloading stateful storage to external drivers

آکادمی همراه‌اول

# Docker Volumes (cont'd)

| Command | Purpose |
| --- | --- |
| docker volume create myvolume | Create a new named volume |
| docker volume ls | List all volumes |
| docker volume inspect myvolume | Show detailed info (driver, mountpoint, usage) |
| docker volume rm myvolume | Remove a specific volume |
| docker volume prune | Remove all unused volumes |
| docker run -v myvolume:/app/data nginx | Mount a volume inside a container |

آکادمی همراه‌اول

# Bind Mounts

- Mounts a host path directly into the container (docker run -v /host/path:/container/path)
- Use cases: dev workflows, direct access to host data, device access
- Pros: no copy, immediate visibility on host; Cons: couples container to host, permission/UID issues
- SELinux/AppArmor: must use :z or :Z labels for shared contexts on RHEL/SELinux systems
- Performance: native filesystem performance; beware NFS semantics with file locking & O_SYNC

آکادمی همراه اول

# Bind Mounts (cont'd)

| Description | Example |
| --- | --- |
| Basic bind mount: mounts a host directory into the container | docker run -v /home/user/data:/app/data myimage |
| More explicit syntax for bind mounts (preferred for complex cases) | docker run --mount type=bind,source=/home/user/data,target=/app/data myimage |
| Mount as read-only | docker run -v /home/user/data:/app/data:ro myimage |
| Mount as read-write (default) | docker run -v /home/user/data:/app/data:rw myimage |
| View details of bind mounts used in a container | docker inspect mycontainer |

آکادمی همراه‌اول

# tmpfs (Memory-backed) - Details & Tradeoffs

| Aspect | Details |
| --- | --- |
| What it is | A filesystem mounted from RAM (tmpfs) using kernel memory |
| Use cases | Session data, caches, ephemeral tmp files, secrets during runtime |
| Persistence | Volatile — cleared on reboot/container restart |
| Performance | Very fast (RAM) but limited by system memory |
| Limits & sizing | Specify --tmpfs size=... or mount options; watch OOM |
| Security | Reduced disk I/O; sensitive to memory leaks/attacks |

آکادمی همراه‌اول

# Docker Volumes & Drivers

- Named volumes live under Docker's control (/var/lib/docker/volumes/<id>/_data)
- Volume drivers: local, nfs, cloud (EBS plugin), Ceph/RBD, Portworx, Longhorn, RexRay
- Drivers provide features: snapshots, replication, encryption, quotas, access modes
- CSI (Container Storage Interface) enables Kubernetes-style storage plugins; Docker plugins follow plugin API

آکادمی همراه‌اول

# Storage Types - Comparison

| Type | Ephemeral? | Performance | Use cases | Notes |
|---|---|---|---|---|
| Writable layer (overlayfs) | Yes | Fast (local) - COW overhead | Runtime changes, temp files | Shared base layers, container-specific writes go to upperdir |
| tmpfs (ramfs) | Yes | Very fast (RAM) | Sensitive data, caches | Volatile; size limited by memory |
| Bind mount | Depends (host path) | Native host FS perf | Dev, access host devices | Permission mapping, couples to host |
| Named Volume (local) | No (persists) | Local disk speed / driver | Databases, stateful services | Managed by Docker; backup/restore supported |
| Network Volume (NFS/GlusterFS/Contiv) | No (persists) | Network latency dependent | Shared storage, clusters | Requires driver; consider consistency & locking |

# Storage Security - Considerations

| Area | Mitigation / Tip |
| --- | --- |
| UID/GID mapping | Use user namespaces, chown data directory, avoid root in container |
| SELinux/AppArmor | Use proper labels (:z / :Z) and profiles; validate policies |
| Encryption at rest | Use encrypted block devices or plugin features; KMS for keys |
| Access control | Least privilege for volume drivers and API access; audit logs |
| Secrets handling | Avoid baking secrets into images; use tmpfs or secret managers |

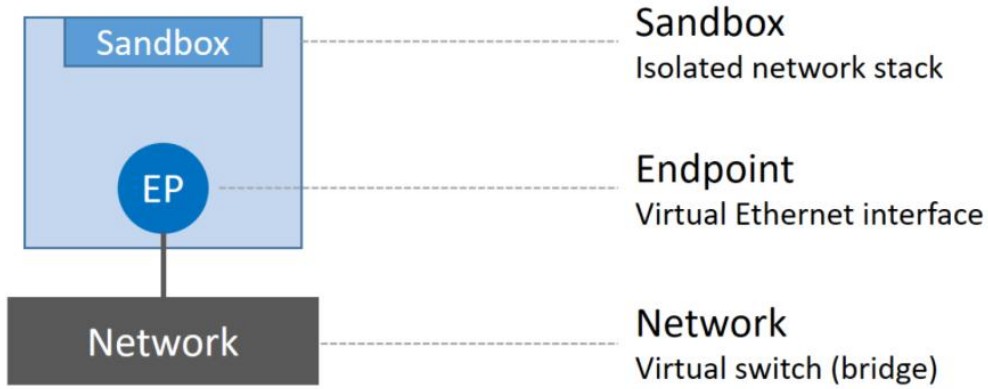
آکادمی همراه‌اول

# Networking - Foundations

- Each container has its own network namespace and virtual interface (veth pair) connected to host bridge or network
- Docker networking drivers implement connectivity & isolation: bridge, host, overlay, macvlan/ipvlan
- iptables/nftables used for NAT and filtering; CNI plugins provide advanced networking

آکادمی همراه اول

# CNM




## What is CNM?

- Docker's **networking architecture** introduced in libnetwork (2015)
- Defines a **modular model** for container connectivity
- Enables pluggable **network drivers** (bridge, overlay, macvlan, 3rd-party)

## Key Components

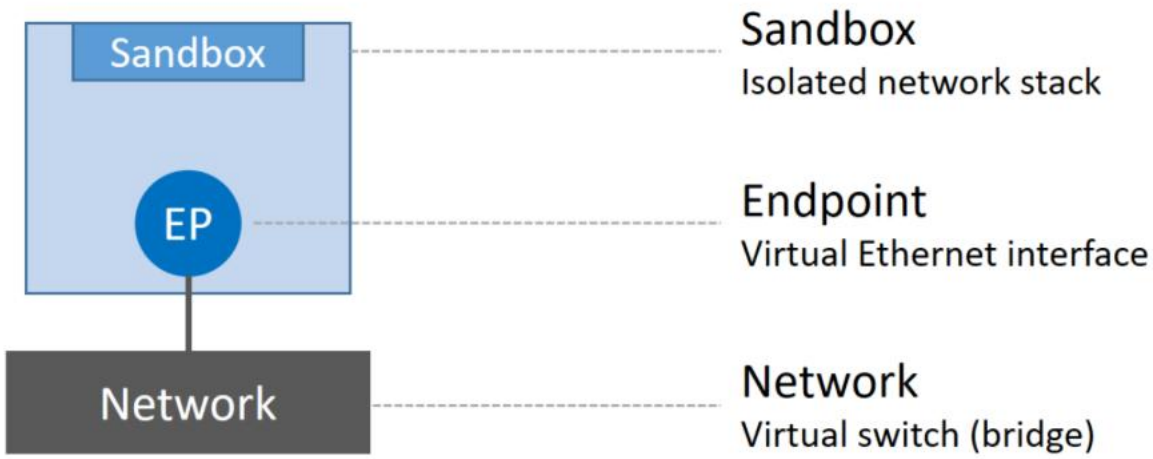
- **Sandbox** → Isolated network stack for a container (interfaces, routes, firewall rules)
- **Endpoint** → Virtual NIC that connects a sandbox to a network
- **Network** → Logical construct that groups endpoints (bridge, overlay, etc.)

آکادمی همراه‌اول

# CNM

## Benefits

- Clear separation of concerns (sandbox ↔ endpoint ↔ network)
- Extensible with 3rd-party CNM-compliant drivers
- Forms the basis for modern Docker networking



آکادمی همراه‌اول

# Networking - overview

- Each container has its own network namespace and virtual interface (veth pair) connected to host bridge or network
- Docker networking drivers implement connectivity & isolation: bridge, host, overlay, macvlan/ipvlan
- iptables/nftables used for NAT and filtering

آکادمی همراه‌اول

# Host Network Driver

- Container shares the **host's network stack**
- No network isolation → container uses host's IP directly
- Lower latency, no NAT overhead
- using--network host
- ⚠️ Reduced security (no isolation between host & container)
- **Use cases: performance-sensitive workloads, network appliances, containerized monitoring agents**

آکادمی همراه‌اول

# MACVLAN

- Assigns a **unique MAC address** to each container
- Container appears as a **physical device** on the LAN
- Bypasses Docker bridge and NAT → **direct Layer 2 connectivity**
- Works like each container is a separate host on the network
- Networking equipment needs to be able to handle "promiscuous mode", where one physical interface can be assigned multiple MAC addresses
- Modes: bridge, vepa, passthru, private
- Pros: containers appear as first-class hosts on network, minimal NAT
- Cons: host cannot easily communicate with macvlan child unless configured; security concerns

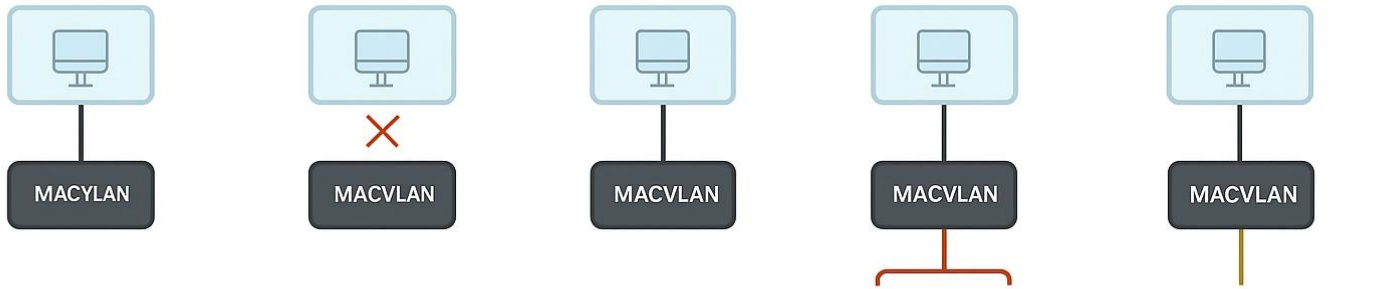
## Use Cases

- When containers need to be **directly accessible on the LAN**
- Integration with **legacy applications** expecting real MAC/IP
- High-performance networking (low overhead, no NAT)

آکادمی همراه‌اول

# MACVLAN

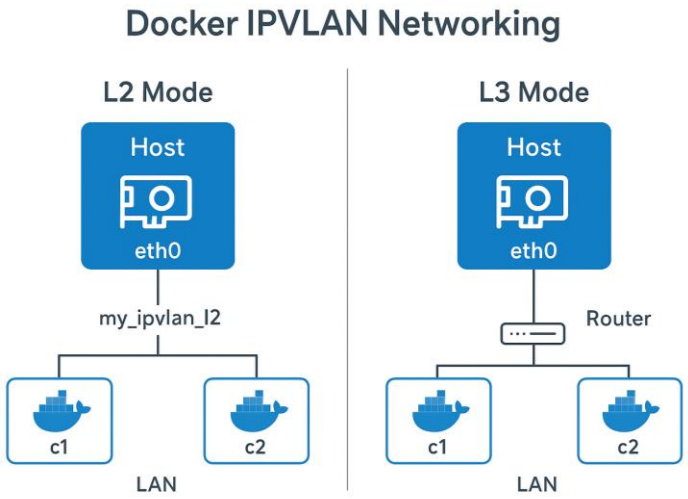
## MACVLAN Modes in Docker




| Bridge | Private | VEPA | Passthru | 802.1Q Trunk |
|---|---|---|---|---|
| Default mode | Containers isolated from each other & host | All traffic sent upstream to physical switch | Container takes over the host NIC's MAC | Containers attached to different VLANs via sub-interfaces |
| Dernisteo | Security isolation | Enterprise setups witch switch monitoring | When container must look | Multi-tenant VLAN separation |

آکادمی همراه‌اول

# IPVLAN




- Similar to **MACVLAN**, but containers share the host's **MAC address**.
- Each container gets a unique **IP address** but uses the **same MAC**.
- Simplifies switch configuration compared to MACVLAN (no flood of MACs).
- IPVLAN operates at L3; two modes: L2 (bridge-like) and L3 (routed)
- Pros: simpler than macvlan for host-container communication, better scalability
- Cons: less mature tooling in some environments, routing complexity
- Use cases: large-scale host networking where MAC exhaustion or policy is a concern

آکادمی همراه‌اول

# Bridge Network (default)

- Default network driver in Docker
- Each container gets its own private IP (via virtual Ethernet pair)
- Containers communicate through a virtual bridge (docker0)
- NAT (Network Address Translation) allows outbound internet access
- Containers on the same bridge can reach each other via IP/hostname
- using --network bridge or not specify since its default
- Use cases: Default choice for most apps, Inter-container communication in isolated environments, Provides network isolation from the host

آکادمی همراه‌اول

# Overlay Networks

- Create encrypted/unencrypted overlay networks across Docker hosts swarm
- Uses VXLAN encapsulation and an overlay mesh or control plane
- Pros: multi-host connectivity, service discovery across nodes
- Cons: MTU overhead (reduce MTU), encapsulation CPU cost, troubleshooting complexity

آکادمی همراه اول

# Networking - Best Practices (Summary)

- Use bridge for dev; macvlan/ipvlan for L2 integration or performance needs
- Prefer overlay/CNI solutions for multi-host orchestration (Kubernetes, Swarm)
- Tune MTU when using overlay networks; monitor encapsulation overhead
- Apply network policies and least-privilege; encrypt traffic between hosts when needed
- Use observability tools (eBPF, tcpdump, ss) to locate bottlenecks and failures

آکادمی همراه‌اول

# Network Performance - Tradeoffs

| Driver/Mode | Latency | Throughput | MTU/Notes |
|---|---|---|---|
| host | Lowest | Highest | No NAT; uses host MTU |
| bridge | Low | Good (NAT overhead) | Default MTU; NAT overhead |
| macvlan | Low | High (native L2) | Requires MACs; host to child caveats |
| overlay (VXLAN) | Higher | Good (encapsulation cost) | Lower effective MTU; adjust MTU |
| ipvlan | Low | High | Routing depends on mode |

آکادمی همراه‌اول

# Review

| Command | Description | Example |
|---|---|---|
| docker network ls | List all networks | docker network ls |
| docker network inspect <network> | Show details of a network | docker network inspect bridge |
| docker network create <name> | Create a new network (default bridge) | docker network create mynet |
| docker network create -d <driver> <name> | Create network with specific driver (bridge, overlay, macvlan, ipvlan) | docker network create -d macvlan mymacvlan |
| docker network rm <name> | Remove a network | docker network rm mynet |
| docker network connect <network> <container> | Connect a container to a network | docker network connect mynet c1 |
| docker network disconnect <network> <container> | Disconnect a container from a network | docker network disconnect mynet c1 |